Noise Is Not Noise When the Policy Chooses It: Unbiased Reinforcement Learning from an Exploitable Judge with Random Audits

Vahit FERYAD *vahit.feryat@gmail.com*
Independent researcher, Istanbul, Türkiye
ORCID: 0000-0002-3282-339X
Code and data: github.com/vahit19/reward-audit/tree/main/audited_rewards

Abstract

Reinforcement-learning post-training of language models increasingly pays the policy with a cheap grader, such as a large language model (LLM) judge, a unit-test harness or a rule-based answer checker. The usual way to handle a grader's mistakes is to treat them as label noise with fixed rates. That view misses what makes them dangerous during training: the policy moves toward the outputs on which the grader is wrong, so the error rate on the policy's own outputs grows as it learns. We model reward hacking as policy-dependent label noise and prove three results. First, any procedure that sees only judge outputs, together with gold labels collected under a distribution fixed before training, can be led to a bad policy, because two problems with opposite optimal policies can produce identical data for it. Second, if a random fraction p of the policy's own outputs is audited after they are written, and the policy is paid $\tilde{r} = g(j) + \xi\,(y - g(j))/p$, where j is the judge's verdict, y the gold label, ξ the audit indicator and g a trust function fitted on past audits, then the leave-one-out policy-gradient estimator is unbiased for the gold objective under every policy and every error pattern of the judge. Third, the price of this guarantee is variance, and we give it exactly: $\mathbb{E}[(1/p - 1)(y - g)^2]$ on top of the variance of the gold reward. The best trust function is the judge's calibration on the current policy's outputs, the best audit allocation is Neyman's, and the same audits drive an alarm that stays valid at any stopping time. In a controlled simulation, judge-only training and fixed-rate noise correction both collapse onto an exploit (true success 0.23 and 0.08), while audited rewards reach 0.684, the same as paying with gold on every sample, with one eighth of the gold labels. At one thirty-second of the labels, audits that ignore the judge start to fail and audited rewards do not. In GRPO training of a 0.5-billion-parameter model on GSM8K, a judge without the answer key reports a mean reward of 0.66 for a policy whose true accuracy is 0.26, while the audited reward reports 0.22 against a true 0.23, using one eighth of the gold labels.

1 Introduction

A grader can agree with experts on a fixed test set and still be wrong exactly where a policy learns to aim. Reports from production reinforcement learning (RL) runs describe policies that exit a test harness early so that the tests appear to pass (MacDiarmid et al., 2025), graders that reward confident guessing over an honest "I don't know" (Kalai et al., 2025), and multi-step strategies that mislead an overseer (Farquhar et al., 2025). We saw the same pattern at small scale. When a 0.5-billion-parameter policy was trained on grade-school math with group relative policy optimization (GRPO; Shao et al., 2024) and paid by a 1.5-billion-parameter judge that had no access to the answer key, the training reward rose from 0.55 to 0.73 over 100 steps, while the judge accepted 43% of the wrong answers in the final steps (Feryad, 2026).

The standard response treats grader errors as noise with fixed false-positive and false-negative rates, estimates the rates on a held-out set, and corrects the reward (Natarajan et al., 2013; El Mansouri et al., 2025). The

correction is right when the error rates do not depend on what the policy does. Reward hacking is precisely the case in which they do. In our simulation the judge's precision on the policy's own outputs falls from 0.81 to 0.25 during training (Figure 3). A correction calibrated before training keeps trusting the judge as much as it did at the start, and it therefore drives the policy into the exploit faster than the raw judge does.

We take a different view of the grader. It is a cheap predictor of a gold reward that the policy may learn to exploit, and a small number of gold labels, placed at random on the policy's own outputs after they are written, is enough to keep the training signal honest. The idea is the logic of a tax audit (Becker, 1968): if any return may be audited with probability p and an audited discrepancy is settled at $1/p$ times its size, misreporting has zero expected value, whoever chooses what to report. Figure 1 shows one training step under this scheme.

Contributions.

1. An impossibility result. Corrections that use only judge outputs, or gold labels from a distribution fixed before training, cannot rule out reward hacking. We construct two problems that such a method cannot tell apart and whose optimal policies differ (Proposition 1).
2. An unbiasedness guarantee. The audited reward, a Horvitz–Thompson correction (Horvitz & Thompson, 1952) of the judge used inside the policy gradient, gives an unbiased gradient of the gold objective under every policy and every error pattern of the judge (Theorem 2).
3. An exact price. We give the variance cost of auditing in closed form, derive the optimal trust function and the optimal audit allocation, and show that a judge is worth exactly the share of the gold reward's second moment it explains on the policy's own outputs (Theorem 3).
4. A monitor. The same audits drive a test of the judge's false-accept rate that may be checked after every step without inflating its error rate (Proposition 4).
5. Experiments. A controlled bandit shows the failure of judge-only and fixed-rate corrections and the recovery by audits, and confirms the variance formula numerically. In GRPO training on GSM8K, the audited reward stays within 0.01 of the true accuracy with one eighth of the gold labels, while the judge alone overstates it by 0.40 (§5).

The estimator itself is classical. It was recently used at evaluation time to identify the best of several systems with an LLM judge and a limited number of human labels (Ao et al., 2026). What is new here is its analysis at training time, where the learner chooses the distribution on which the judge is evaluated and has an incentive to choose it badly.

2 Related work

Reward hacking. Skalse et al. (2022) defined reward hacking formally, and Gao et al. (2023) measured how the gap between a learned reward model and the gold reward grows with optimization. MacDiarmid et al. (2025) showed that hacks learned in production coding environments generalize to broader misalignment. MONA restricts the optimization horizon so that multi-step hacks are not learned even when the overseer cannot detect them (Farquhar et al., 2025). Adversarial reward auditing trains an auditor model that looks for exploits and down-weights them (Beigi et al., 2026); the auditor is itself a learned grader and comes without an unbiasedness guarantee. Our results are complementary to these methods. They make single-step judge errors unprofitable in expectation and detectable, at a label cost that can be computed in advance.

Learning from noisy rewards. Learning with class-conditional label noise corrects a loss using known flip rates (Natarajan et al., 2013). Noise-corrected GRPO brings the same correction to RL with verifiable rewards and estimates the rates on a held-out calibration set (El Mansouri et al., 2025). Both assume flip rates that do not depend on the policy, an assumption that Proposition 1 shows is decisive when the policy can choose where the grader errs.

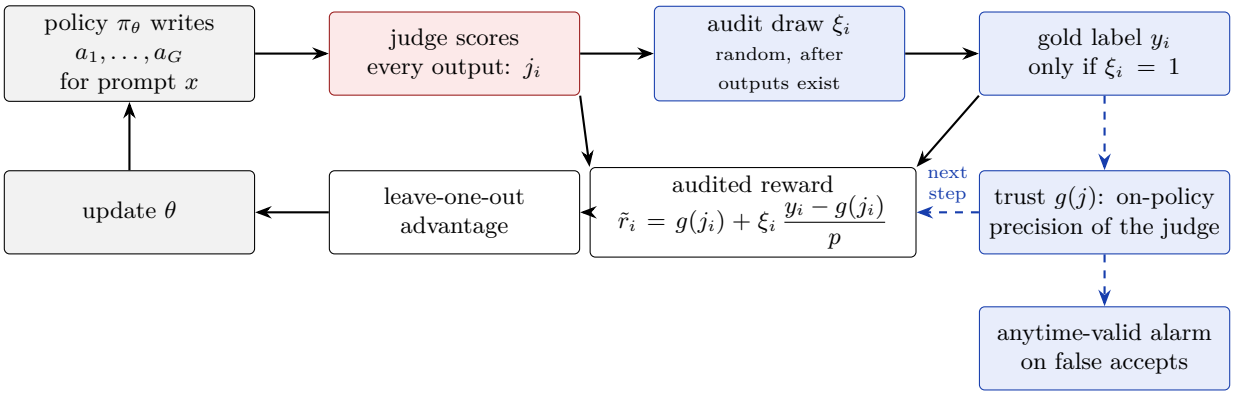


Figure 1: One training step with audited rewards. The judge scores every output; a random subset, drawn only after the outputs exist, receives a gold label. Each output is paid the judge's calibrated score $g(j)$, and audited outputs are paid an extra correction scaled by $1/p$. The audits also refit the trust function used at the next step and feed a sequential alarm (dashed). Red: the exploitable judge; blue: components that use gold labels.

Table 1: How training-time approaches to an imperfect grader compare. "Unbiased under exploitation" means the expected training signal of every policy equals its gold value, whatever the judge's errors.

Approach	Gold labels used	Unbiased under exploitation	Detects drift
Judge only	none	no	no
Fixed-rate noise correction	held-out set, before training	no (Prop. 1)	no
Learned auditor	none at training time	no guarantee	partly
Gold label on every sample	every output	yes	not needed
Audited reward (ours)	**random fraction p of outputs**	**yes (Thm. 2)**	**yes (Prop. 4)**

Statistics with a cheap predictor and a few gold labels. The audited reward is the Horvitz–Thompson estimator (Horvitz & Thompson, 1952) with the judge as a control variate, the same structure as prediction-powered inference (Angelopoulos et al., 2023) and active statistical inference (Zrnic & Candès, 2024), and the optimal audit rate is a Neyman allocation (Neyman, 1934). Ao et al. (2026) use inverse-propensity-weighted residuals of an LLM judge to identify the best system at evaluation time. The alarm uses testing by betting (Waudby-Smith & Ramdas, 2024) and Ville's inequality (Ville, 1939). These tools assume that the population being estimated is fixed. In RL training the population is the policy's output distribution, which changes with every update and is pushed toward the judge's errors.

Table 1 places the main approaches side by side.

3 Setup

Prompts are drawn as $x \sim D$, and the policy $\pi_\theta(a \mid x)$ produces an output a. Each pair (x, a) has a gold reward $y \in [0, 1]$, which is expensive to obtain (a verified answer or an expert's label), and a judge output j, which is cheap. The joint law $P(y, j \mid x, a)$ is arbitrary and unknown; we call it the judge's error pattern. The objective is

$$J(\theta) = \mathbb{E}_{x \sim D}\, \mathbb{E}_{a \sim \pi_\theta(\cdot|x)}\big[\mu(x, a)\big], \qquad \mu(x, a) = \mathbb{E}[y \mid x, a].$$

Definition 1 (Audited reward)**.** At training step t, after a has been sampled and j observed, the learner draws an audit indicator $\xi \in \{0, 1\}$ and observes y only if $\xi = 1$. The audit rate $p = p_t(x, a, j) \geq p_{\min} > 0$ and the trust function $g = g_t(x, a, j)$ are *predictable*: they may depend on everything observed before step t but not on the current gold label. The audit is *blind*: $P(\xi = 1 \mid y, x, a, j, \mathcal{F}_{t-1}) = p$. The audited reward is

$$\tilde{r} \;=\; g \;+\; \frac{\xi}{p}\,(y - g). \tag{1}$$

Table 2: Notation, with the concrete objects used in the GRPO experiment.

Symbol	Meaning	In the GRPO experiment
π_θ	policy being trained	Qwen2.5-0.5B-Instruct
j	judge verdict, cheap	Qwen2.5-1.5B-Instruct, no answer key, yes/no
y	gold reward, expensive	exact match of the final number with GSM8K
ξ	audit indicator	one of 8 completions per prompt, uniform
p	audit rate	1/8
$g(j)$	trust in the judge	running $P(y = 1 \mid j)$ from past audits
G	samples per prompt	8

Setting $g = 0$ gives judge-free audits, $\tilde{r} = \xi y/p$, and $p = 1$ gives gold on every sample. Blindness holds for independent Bernoulli draws, and also for the design used in our experiments, where exactly one of the G outputs for a prompt is chosen uniformly at random after they are generated, so that $p = 1/G$. Table 2 summarizes the notation.

4 Theory

4.1 Why fixed corrections fail

Proposition 1 (Judge-only and off-policy corrections can be fooled)**.** *Let $\mathcal{A}$ be any training procedure whose data are judge outputs on the policy's own samples and gold labels on pairs (x, a) drawn from a distribution Q fixed before training. For every $\Delta \in (0, \frac{1}{2}]$ there are two problems M and M' in which $\mathcal{A}$ receives identically distributed data, whose optimal policies differ, and for which the policy $\hat{\pi}$ returned by $\mathcal{A}$ satisfies*

$$\max\{J^*_M - J_M(\hat{\pi}),\ J^*_{M'} - J_{M'}(\hat{\pi})\} \ \geq\ \Delta/2.$$

Proof. Take one prompt and two outputs, s (solve) and h (hack). In both problems $j \mid s \sim \mathrm{Bern}(1/2)$ and $j \mid h \sim \mathrm{Bern}(0.9)$, independently of y, and Q puts no mass on h. Gold values are $\mu(s) = 1/2$ in both problems, $\mu_M(h) = 1/2 + \Delta$ and $\mu_{M'}(h) = 1/2 - \Delta$. Every observation available to $\mathcal{A}$ has the same law in M and in M', so $\hat{\pi}$ has the same law in both. If q is the expected mass that $\hat{\pi}$ puts on h, the suboptimality is $(1 - q)\Delta$ in M and $q\Delta$ in M', and the larger of the two is at least $\Delta/2$. □

Fixed-rate noise correction, which estimates false-positive and false-negative rates on a calibration set, is a special case of $\mathcal{A}$. If Q puts mass η on h instead of zero, Le Cam's two-point method (Tsybakov, 2009) shows that on the order of $1/(\eta\Delta^2)$ gold labels are needed before M and M' can be told apart. Gold labels are needed where the policy is going, and a hack is, by definition, where the policy is going.

4.2 Audits remove the bias

Theorem 2 (Audited rewards are unbiased under every policy)**.** *At every step, $\mathbb{E}[\tilde{r} \mid \mathcal{F}_{t-1}, x, a, j] = \mathbb{E}[y \mid x, a, j]$, hence $\mathbb{E}[\tilde{r} \mid x, a] = \mu(x, a)$. With $G \geq 2$ samples per prompt and the leave-one-out estimator (Kool et al., 2019; Ahmadian et al., 2024)*

$$\hat{\nabla} = \frac{1}{G}\sum_{i=1}^{G}\Big(\tilde{r}_i - \frac{1}{G-1}\sum_{k\neq i}\tilde{r}_k\Big)\nabla_\theta \log \pi_\theta(a_i \mid x),$$

we have $\mathbb{E}[\hat{\nabla}] = \nabla J(\theta)$ for every θ and every error pattern. In particular, the expected audited reward of any policy equals its gold value, so no policy can raise its expected training signal by exploiting the judge.

Proof. Given $(\mathcal{F}_{t-1}, x, a, j)$, the quantities g and p are fixed and $\mathbb{E}[\xi \mid y, \cdot] = p$, so $\mathbb{E}[\xi(y - g)/p \mid \cdot] = \mathbb{E}[y \mid x, a, j] - g$. Adding g gives the first claim, and the tower rule gives $\mu(x, a)$. By the score-function identity, $\mathbb{E}[\tilde{r}_i \nabla \log \pi_\theta(a_i \mid x)] = \mathbb{E}[\mu(x, a_i)\nabla \log \pi_\theta(a_i \mid x)] = \nabla J(\theta)$. For $k \neq i$, conditioning on all outputs and judge

verdicts gives $\mathbb{E}[\tilde{r}_k \mid \cdot] = \mathbb{E}[y_k \mid x, a_k, j_k]$, which does not depend on a_i given x, while $\mathbb{E}[\nabla \log \pi_\theta(a_i \mid x) \mid x] = 0$. The baseline terms therefore vanish in expectation. □

The only design rule is that the audit decision is made after the output exists and does not depend on its gold label. The policy then has no way to behave differently on audited outputs. GRPO (Shao et al., 2024) also divides advantages by the standard deviation of the group's rewards. This rescales each update and introduces a small bias, so Theorem 2 is stated for the leave-one-out form, and our GRPO experiment keeps the standard GRPO normalization.

4.3 The price of auditing

Theorem 3 (Variance, optimal trust and optimal allocation)**.** *For every policy,*

$$\mathrm{Var}(\tilde{r} \mid x, a) \;=\; \mathrm{Var}(y \mid x, a) \;+\; \mathbb{E}\Big[\Big(\frac{1}{p} - 1\Big)(y - g)^2 \;\Big|\; x, a\Big]. \tag{2}$$

(a) For fixed p, the overhead is minimized by $g^* = \mathbb{E}[y \mid \text{inputs of } g]$. If g sees only the judge verdict, $g^*(j) = \mathbb{E}_\pi[y \mid j]$, the judge's calibration on the current policy's outputs; for a yes/no judge, $g^*(\text{yes})$ is its on-policy precision.

(b) Under a budget $\mathbb{E}_\pi[p] \le B$, the overhead $\mathbb{E}[(1/p - 1)\, s^2]$ with $s^2 = \mathbb{E}[(y - g)^2 \mid x, a, j]$ is minimized by $p^* = \min(1, s/\tau)$, with τ chosen to meet the budget (Neyman allocation).

(c) For a constant rate p, the overhead with $g = g^*$ equals the overhead of judge-free audits multiplied by $\mathbb{E}[(y - g^*)^2]/\mathbb{E}[y^2]$. A judge is worth exactly the share of the gold reward's second moment that it explains on the policy's own outputs.

Proof. Write $m = \mathbb{E}[y \mid x, a, j]$. Because $\xi^2 = \xi$ and $\mathbb{E}[\xi \mid \cdot] = p$, $\mathbb{E}[\tilde{r}^2 \mid x, a, j] = g^2 + 2g(m - g) + \mathbb{E}[(y - g)^2 \mid \cdot]/p$, and therefore $\mathrm{Var}(\tilde{r} \mid x, a, j) = \mathbb{E}[(y-g)^2 \mid \cdot]/p - (m-g)^2$. Using $\mathbb{E}[(y-g)^2 \mid \cdot] = \mathrm{Var}(y \mid x, a, j) + (m-g)^2$, this equals $\mathrm{Var}(y \mid x, a, j) + (1/p - 1)\,\mathbb{E}[(y - g)^2 \mid x, a, j]$. The law of total variance, together with $\mathbb{E}[\tilde{r} \mid x, a, j] = m$, adds $\mathrm{Var}(m \mid x, a)$ to both $\mathrm{Var}(\tilde{r} \mid x, a)$ and $\mathrm{Var}(y \mid x, a)$, which gives (2). Part (a) is the least-squares property of the conditional mean. For (b), the Lagrangian $s^2/p + \tau^2 p$ is minimized pointwise at $p = s/\tau$. Part (c) follows by substituting $g = g^*$ and $g = 0$ into the overhead. □

Theorem 3 changes what exploitation costs. Under audited rewards, a policy that drifts toward outputs the judge wrongly accepts no longer gains reward in expectation; it only adds variance. As the drift happens, $g^*(\text{yes})$ falls on its own, so trust in the judge decays as fast as the judge is exploited (Figure 3).

4.4 An alarm from the same audits

Proposition 4 (Anytime-valid alarm)**.** *Let $Z_k = \mathbf{1}\{\text{judge accepts and } y = 0\}$ on the k-th audited output, and let H_0 state that $\mathbb{E}[Z_k \mid \text{past}] \le \varepsilon$ for all k. For predictable bets $\lambda_k \in [0, 1/\varepsilon)$, the wealth $W_n = \prod_{k \le n}\big(1 + \lambda_k(Z_k - \varepsilon)\big)$ is a nonnegative supermartingale under H_0, so $P(\exists n : W_n \ge 1/\alpha) \le \alpha$.*

This is Ville's inequality (Ville, 1939) applied to a betting process (Waudby-Smith & Ramdas, 2024). The alarm may be checked after every step. Because audit rates only need to be predictable, raising p after an alarm keeps Theorem 2 intact.

5 Experiments

5.1 A controlled exploit

Design. One prompt has four answer strategies with gold success rates $\mu = (0.50, 0.20, 0.05, 0.70)$. The judge accepts a correct answer with probabilities $(0.90, 0.90, 0.95, 0.80)$ and a wrong one with $(0.10, 0.10, 0.95, 0.05)$. Strategy 3 is the exploit: the judge accepts it 95% of the time although it is almost never right, and the initial policy plays it only 3% of the time. We train a softmax policy with the

Table 3: Controlled exploit, 200 seeds. True success is the gold value of the policy; the best achievable value is 0.70. Ranges are 2.5th and 97.5th percentiles over seeds.

Reward used for training	Gold labels	Step 750	Step 1500 (range)	Seeds on exploit
Gold on every sample	12,000	0.652	0.684 (0.682–0.687)	0%
Judge only	0	0.528	0.229 (0.133–0.415)	9%
Fixed-rate noise correction	1,500	0.514	0.084 (0.071–0.108)	100%
Audits only, judge ignored	1,500	0.640	0.682 (0.653–0.690)	0%
Audited reward	**1,500**	**0.649**	**0.684 (0.677–0.689)**	**0%**

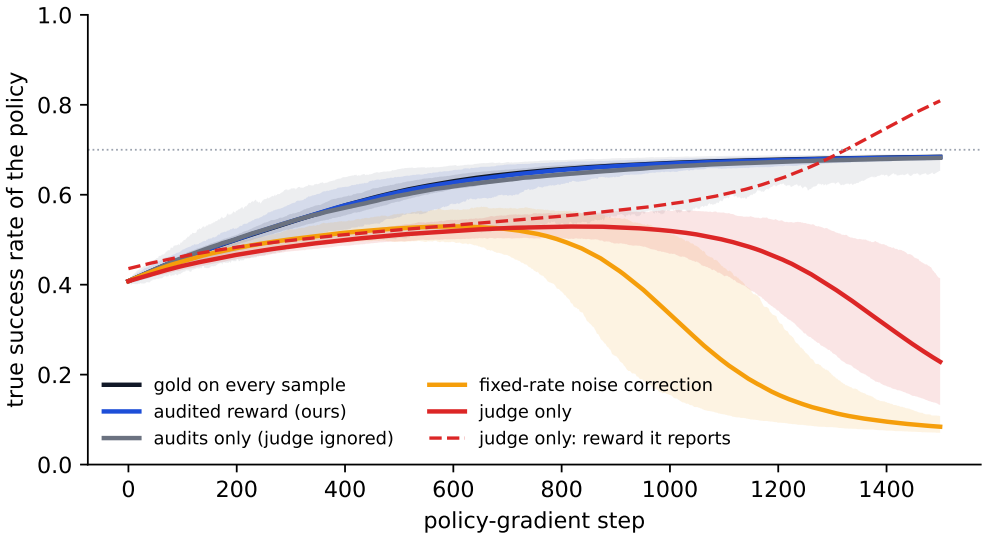


Figure 2: True success of the policy (mean and 95% range over 200 seeds). The judge-only run reports a rising reward (dashed) while its true success collapses. Fixed-rate noise correction collapses faster. Audited rewards match gold on every sample with one eighth of the labels.

leave-one-out policy gradient, $G = 8$ samples per step, for 1,500 steps and 200 seeds. Audited methods audit exactly one of the eight samples, chosen uniformly after sampling ($p = 1/8$). Fixed-rate noise correction receives the same 1,500 gold labels, collected once from the initial policy, and pays $(j - \widehat{\text{FPR}})/(\widehat{\text{TPR}} - \widehat{\text{FPR}})$. The audited trust $g(j)$ is the past audits' estimate of $P(y = 1 \mid j)$ with exponential forgetting. Appendix A lists all settings.

Results. Table 3 and Figure 2 show the outcome. Training on the judge alone reports a rising reward while true success first improves and then collapses to 0.23. Fixed-rate noise correction is a monotone transform of the judge, so it amplifies the judge's preference for the exploit and puts every seed on it. Audited rewards end at 0.684 with one eighth of the gold labels, the same as paying gold on every sample. Figure 3 shows the mechanism: when the judge pays, its precision on the policy's outputs falls from 0.81 to 0.25, and in the audited run the estimated trust $g(\text{yes})$ follows the precision of the audited policy.

The variance formula. At the initial policy with a fixed trust function, the empirical variance of $\tilde{r}$ over 4×10^5 draws is 1.192, and Equation (2) predicts 1.199.

Label budget. We repeat the experiment with independent Bernoulli audits at rates from 1/8 to 1/64, 100 seeds each (Table 4, Figure 4). As labels become scarce, audits that ignore the judge lose both final success and stability, from 0.681 to 0.631, with the share of seeds above 0.6 falling from 99% to 70%. The audited reward stays close to the optimum, from 0.684 to 0.674, with 100% to 95% of seeds above 0.6. With

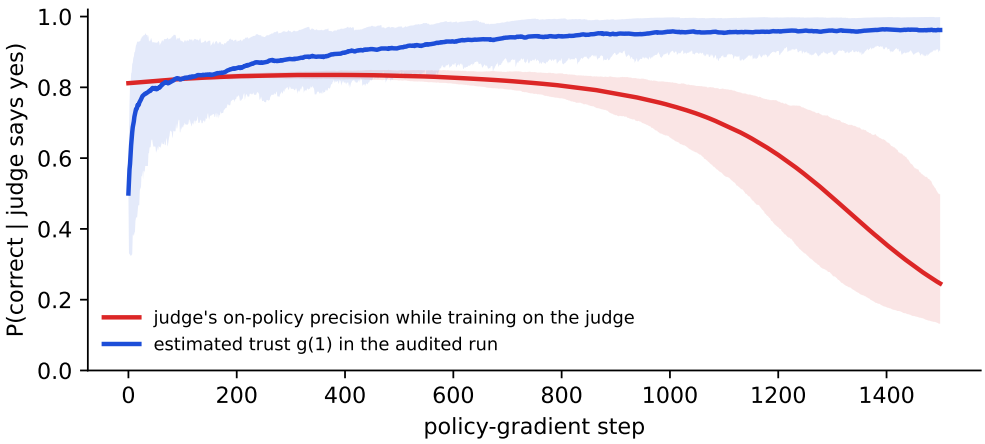


Figure 3: The judge's precision on the policy's own outputs falls from 0.81 to 0.25 when the judge pays the reward. This is the policy-dependent noise that fixed-rate corrections assume away. In the audited run, the estimated trust $g(\text{yes})$ tracks the precision of the audited policy.

Table 4: Label budget, Bernoulli audits, 100 seeds per cell. Final true success (standard deviation over seeds) and share of seeds ending above 0.6.

		Audits only, judge ignored		Audited reward	
Audit rate	Gold labels	Final success	> 0.6	Final success	> 0.6
1/8	1,500	0.681 (0.016)	99%	0.684 (0.003)	100%
1/16	747	0.672 (0.038)	93%	0.684 (0.004)	100%
1/32	375	0.656 (0.059)	84%	0.682 (0.010)	100%
1/64	187	0.631 (0.082)	70%	0.674 (0.034)	95%

375 labels ($p = 1/32$) the audited reward does as well as judge-free audits with 1,500 labels, which is what Theorem 3(c) predicts for a judge that is informative on the policy's outputs.

5.2 GRPO on GSM8K

Design. We train Qwen2.5-0.5B-Instruct (Qwen et al., 2024) on GSM8K (Cobbe et al., 2021) with GRPO for 100 steps, 32 completions per step (four prompts with eight completions each). The judge is Qwen2.5-1.5B-Instruct, asked whether a completion's final answer is correct without seeing the answer key. The gold reward is exact match of the final number with the GSM8K answer. We compare four rewards. Gold on every sample uses 3,200 gold labels. The judge alone uses none. The audited reward and judge-free audits each audit one of the eight completions per prompt, chosen uniformly after generation, for 400 gold labels. The gold-only and judge-only runs are the ones reported by Feryad (2026), with identical settings. Appendix B lists the hyperparameters.

Results. Table 5 and Figure 5 give the outcome. In this run the judge did not pull the policy away from the gold objective. All four rewards end at a true accuracy between 0.27 and 0.30, and with one seed and 320 completions per ten-step window these differences are within sampling noise (standard error about 0.025). The rewards differ in what they tell the trainer. Paid by the judge alone, the trainer sees a mean reward of 0.66 while the policy's true accuracy is 0.26, an overstatement of 0.40 that persists through training. In the audited run the judge accepts between 52% and 66% of completions while only 15% to 30% are correct (Figure 5c), and the trust $g(\text{yes})$ estimated from the audits stays between 0.26 and 0.40, close to the judge's low precision on this policy.

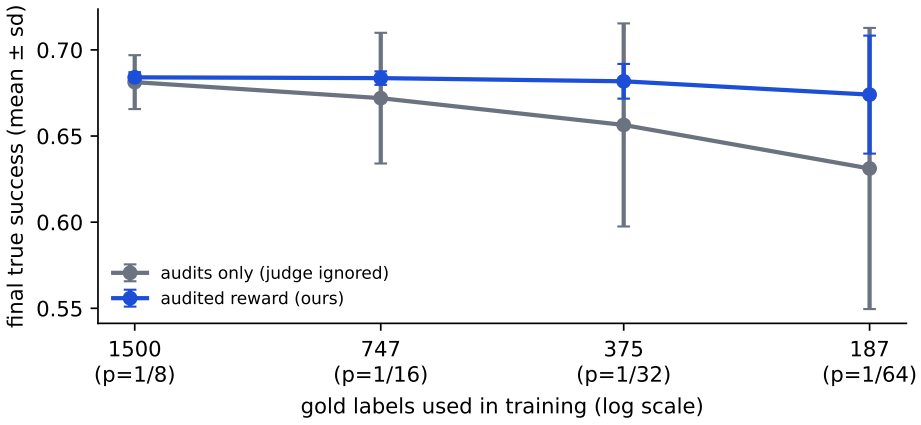


Figure 4: Final true success (mean ± standard deviation over 100 seeds) against the number of gold labels used in training, on a log scale.

Table 5: GRPO on GSM8K, one seed, 100 steps. True accuracy is averaged over the first and last ten steps; the last two columns average over all 100 steps.

		True accuracy		All steps	
Reward used for training	Gold labels	First 10	Last 10	Reward seen	True accuracy
Gold on every sample	3,200	0.219	0.300	0.255	0.255
Judge only	0	0.222	0.303	**0.657**	0.255
Audits only, judge ignored	400	0.216	0.303	0.225	0.234
Audited reward	400	0.200	0.269	0.222	0.228

Both audited rewards are unbiased in practice. Over 400 reward calls, the mean of the reward minus the true accuracy is -0.006 for the audited reward and -0.009 for judge-free audits, and each uses 400 gold labels instead of 3,200. Because the judge explains little of the gold variation here, Theorem 3(c) predicts only a modest gain from using it. The measured gain is of that size: the variance of the reward's error, per completion, is 0.686 for the audited reward and 0.857 for judge-free audits, a reduction of 20%.

6 Discussion

The central point is that a grader's errors are not a fixed property of the grader. They are a property of the grader together with the distribution it is applied to, and in RL that distribution is chosen by an optimizer with an incentive to find the errors. Proposition 1 turns this into a lower bound: no amount of judge data and no calibration set collected in advance can separate a real improvement from a hack the calibration set never saw. Theorem 2 shows that the fix does not require a better judge, only a small amount of gold placed by randomization after the policy has committed to its outputs.

The variance identity makes the trade-off concrete for a practitioner. The audit rate sets a variance multiplier of order $1/p$, and the judge reduces it by the share of gold variation it explains on the current policy. A judge that is good but exploitable is therefore still useful under audits: while it is accurate, it lets audits be rare, and once it is exploited its trust falls and the audits take over, without any change to the training code. The same audits estimate the judge's on-policy precision, which is a direct and interpretable measure of how much the judge is being gamed, and Proposition 4 turns that estimate into an alarm with a guaranteed false-alarm rate.

Three practical rules follow. Audit decisions must be made after generation and must not depend on anything the policy can observe. Trust must be refitted on recent audits rather than fixed at the start. Audit rates

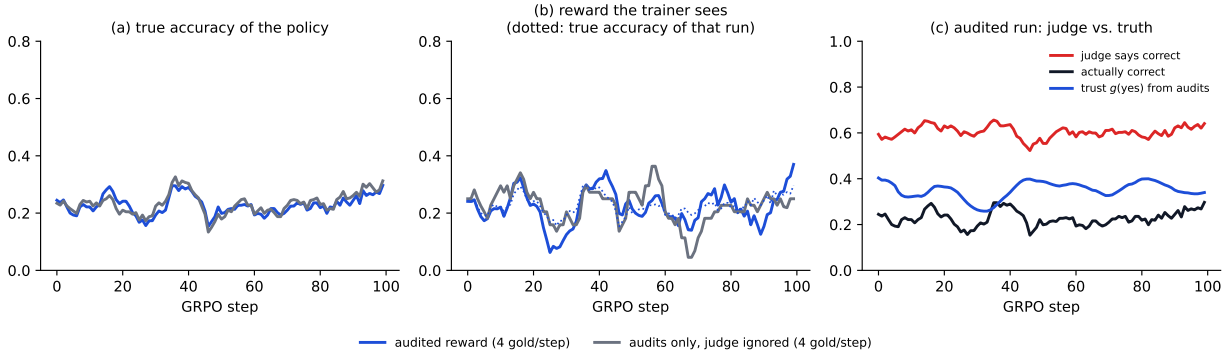


Figure 5: GRPO on GSM8K, 11-step moving averages. (a) True accuracy of the policy under the four rewards. (b) The reward the trainer sees; dotted lines show the true accuracy of the judge-only and audited runs. The judge-only reward sits about 0.4 above the truth, while both audited rewards track it. (c) In the audited run, the share of completions the judge accepts, the share that are correct, and the trust $g(\text{yes})$ that the audits assign to an acceptance.

may be raised wherever the judge and gold disagree most, which is Neyman's rule, and they may change at any time as long as they are set before the gold label is seen.

7 Limitations

Unbiasedness is relative to the gold labeller. If the gold labels are themselves wrong, the method is unbiased for the labeller and not for the truth. The variance cost grows as $1/p$, so very low audit rates need a judge that is informative on the policy's own outputs. GRPO's standard-deviation scaling breaks exact unbiasedness, although the bias is a rescaling of each group's update. The simulation is a stylized four-armed bandit chosen to make the exploit explicit. The GRPO experiment uses a small policy, one task, one seed and 100 steps; in it the judge-only run did not lose true accuracy relative to gold, so the experiment tests label efficiency and the honesty of the reward signal rather than the rescue of a collapsing run. We have not tested multi-step agentic tasks, where an audit may need to cover a whole trajectory.

8 Conclusion

Reward hacking is label noise that the policy chooses. Corrections that fix the noise model before training cannot see it, and our impossibility result makes this precise. Randomly auditing a fraction of the policy's own outputs after they are written, and paying the judge's calibrated score plus an inverse-probability correction, removes the bias for every policy at an exact and computable variance cost. In a controlled exploit, this recovers the performance of full gold supervision with one eighth of the labels and stays stable at one thirty-second, where judge-free audits begin to fail. In GRPO on GSM8K, it keeps the training reward within 0.01 of the true accuracy where the judge alone overstates it by 0.40.

Reproducibility statement

All code, logs and figures are in the `audited_rewards/` folder of the public repository, and all paths below are relative to it. The simulation (`sim/bandit_sim.py`, `sim/sweep.py`) runs on a laptop CPU in minutes and regenerates Tables 3 and 4 and Figures 2–4 from fixed seeds. The GRPO runs use one RTX 4090 for 40 to 50 minutes each, and the two audited runs together cost about 1.3 US dollars of cloud time; the training script logs per-step true accuracy, reward, judge verdicts and trust values, and `llm/analyze.py` produces Table 5 and Figure 5 from the logs.

Declaration of generative AI use

The scientific contribution of this work is the author's own: the research idea, the questions studied, the study design and the interpretation of the results. Under the author's direction, a large language model (Claude, Anthropic) was used as a tool to implement and run the experiments, to draft and edit the manuscript text, and to script the verification of the reference list and the reported numbers. The author reviewed all content and takes full responsibility for the published article.

Broader impact statement

The method is meant to make RL post-training more honest at low cost by preventing a model from being rewarded for fooling its grader. Its guarantee is only as good as the gold labels, and a developer could audit with a weak gold labeller and claim the guarantee anyway. We recommend reporting the gold source, the audit rate and the on-policy precision of the judge alongside any result trained with audited rewards.

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic. Prediction-powered inference. *Science*, 382(6671):669–674, 2023.

Ruicheng Ao, Hongyu Chen, Siyang Gao, Hanwei Li, and David Simchi-Levi. Best arm identification with LLM judges and limited human. *arXiv preprint arXiv:2601.21471*, 2026.

Gary S. Becker. Crime and punishment: An economic approach. *Journal of Political Economy*, 76(2): 169–217, 1968.

Mohammad Beigi, Ming Jin, Junshan Zhang, Qifan Wang, and Lifu Huang. Adversarial reward auditing for active detection and mitigation of reward hacking. *arXiv preprint arXiv:2602.01750*, 2026.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Omar El Mansouri, Fathinah Asma Izzati, Mohamed El Amine Seddik, and Salem Lahlou. Noise-corrected GRPO: From noisy rewards to unbiased gradients. *arXiv preprint arXiv:2510.18924*, 2025.

Sebastian Farquhar, Vikrant Varma, David Lindner, David Elson, Caleb Biddulph, Ian Goodfellow, and Rohin Shah. MONA: Myopic optimization with non-myopic approval can mitigate multi-step reward hacking. *arXiv preprint arXiv:2501.13011*, 2025.

Vahit Feryad. reward-audit: GRPO runs, checker canaries and an anytime-valid audit that flags a bad reward. `https://github.com/vahit19/reward-audit`, 2026.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

Daniel G. Horvitz and Donovan J. Thompson. A generalization of sampling without replacement from a finite universe. *Journal of the American Statistical Association*, 47(260):663–685, 1952.

Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate. *arXiv preprint arXiv:2509.04664*, 2025.

Wouter Kool, Herke van Hoof, and Max Welling. Buy 4 REINFORCE samples, get a baseline for free! In *ICLR Workshop on Deep Reinforcement Learning Meets Structured Prediction*, 2019.

Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural emergent misalignment from reward hacking in production RL. *arXiv preprint arXiv:2511.18397*, 2025.

Nagarajan Natarajan, Inderjit S. Dhillon, Pradeep Ravikumar, and Ambuj Tewari. Learning with noisy labels. In *Advances in Neural Information Processing Systems*, 2013.

Jerzy Neyman. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4):558–625, 1934.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022.

Alexandre B. Tsybakov. *Introduction to Nonparametric Estimation*. Springer, 2009.

Jean Ville. *Étude critique de la notion de collectif*. Gauthier-Villars, Paris, 1939.

Ian Waudby-Smith and Aaditya Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(1):1–27, 2024.

Tijana Zrnic and Emmanuel J. Candès. Active statistical inference. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

A Simulation settings

Setting	Value
Gold success per strategy μ	$(0.50, 0.20, 0.05, 0.70)$
Judge true-positive rate per strategy	$(0.90, 0.90, 0.95, 0.80)$
Judge false-positive rate per strategy	$(0.10, 0.10, 0.95, 0.05)$
Initial logits	$(1.0, 1.0, -1.5, 0.3)$; exploit mass about 3%
Samples per step G	8
Steps, learning rate	1,500, 0.05
Seeds	200 (main), 100 per cell (label budget)
Audit design (main)	one of G samples, uniform, after sampling; $p = 1/8$
Audit design (label budget)	independent Bernoulli, $p \in \{1/8, 1/16, 1/32, 1/64\}$
Trust $g(j)$	running $P(y = 1 \mid j)$, forgetting factor 0.98, prior weight 2
Fixed-rate correction	rates estimated from 1,500 gold labels on the initial policy

B GRPO settings

Setting	Value
Policy	Qwen2.5-0.5B-Instruct
Judge	Qwen2.5-1.5B-Instruct, no answer key, yes/no verdict
Data	GSM8K training split, shuffled with the run seed
Steps	100
Completions per step	32 (4 prompts $\times$ 8 completions)
Learning rate, KL coefficient β	5×10^{-6}, 0.04
Maximum completion length	200 tokens
Audits	one of 8 completions per prompt, uniform, after generation
Trust $g(j)$	running $P(y = 1 \mid j)$, forgetting factor 0.98, prior weight 2
Hardware	one NVIDIA RTX 4090